VoiceOps AI — Capstone Report

From Tokens to Transformers: A Course on GenAI x LLM Mastery · IIT Guwahati × Gnani.ai

1. Problem and objective

ZentroMart receives repetitive questions about delivery, cancellation, returns, refunds and warranties. This project builds a conversational support agent grounded in supplied company documents and synthetic order data, with a voice path using Gnani STT/TTS.

2. Four implementation steps

Step 1 — LLM and prompting: system prompt sets the support role and instructs the model not to invent missing facts or claim tools succeeded without confirmation.

Step 2 — RAG: Markdown documents are split into section chunks and indexed using TF-IDF. Retrieved sections are passed to the LLM and source IDs are retained. Superseded documents are excluded. TF-IDF is a lexical baseline; semantic embeddings are a future improvement.

Step 3 — Conversation and tools: recent conversation turns are retained. Order information is disclosed only after a phone-suffix or delivery-pincode check through the supplied mock tool. Ticket creation is confirmed only when the tool returns success.

Step 4 — Voice: Gnani STT transcribes input audio, the same agent responds, and Gnani TTS generates spoken output. The batch runner saves transcripts, replies, audio and source IDs.

3. Evaluation

The supplied dataset includes 52 single questions and 23 multi-turn conversations (96 customer turns). Run the practice data and replace the placeholders below with measured results. No scores have been fabricated.

Measure	Prompt-only	RAG + tools
Grounded answers	Not measured	Not measured
Retrieval evidence	N/A	Not measured
Conversation/tool decisions	Not measured	Not measured
Voice end-to-end	N/A	Not measured

4. Safety and limitations

Use only the synthetic dataset. Do not commit API keys; keep them in .env. The supplied verification tool is an educational simulation, not production authentication. Ticket creation is local CSV logging. Voice requires a valid Gnani account and compatible SDK. LLM answers require an API key.

5. Future work

Add semantic embeddings and a vector database; automate scoring on practice questions and conversations; strengthen policy-driven routing and date rules; add browser audio recording/playback; test multilingual behaviour and unsupported claims.

6. Run instructions

Extract the supplied dataset beside the project. Install requirements.txt, copy .env.example to .env and configure keys. Validate with the dataset's validate_dataset.py. Launch the chat UI using streamlit run app.py. Batch voice mode: python run.py <audio_folder>.